

决策树

【关键词】树，熵，信息增益

决策树的优缺点

优点：计算复杂度不高，输出结果易于理解，对中间值的缺失不敏感，可以处理不相关特征数据。既能用于分类，也能用于回归

缺点：可能会产生过度匹配问题。

一、决策树的原理

我们经常使用决策树处理**分类问题**。近来的调查表明决策树也是最经常使用的数据挖掘算法。它之所以如此流行，一个很重要的原因就是使用者基本上不用了解机器学习算法，也不用深究它是如何工作的。

决策树分类的思想类似于找对象。现想象一个女孩的母亲要给这个女孩介绍男朋友，于是有了下面的对话：

女儿：多大年纪了？

母亲：26。

女儿：长的帅不帅？

母亲：挺帅的。

女儿：收入高不？

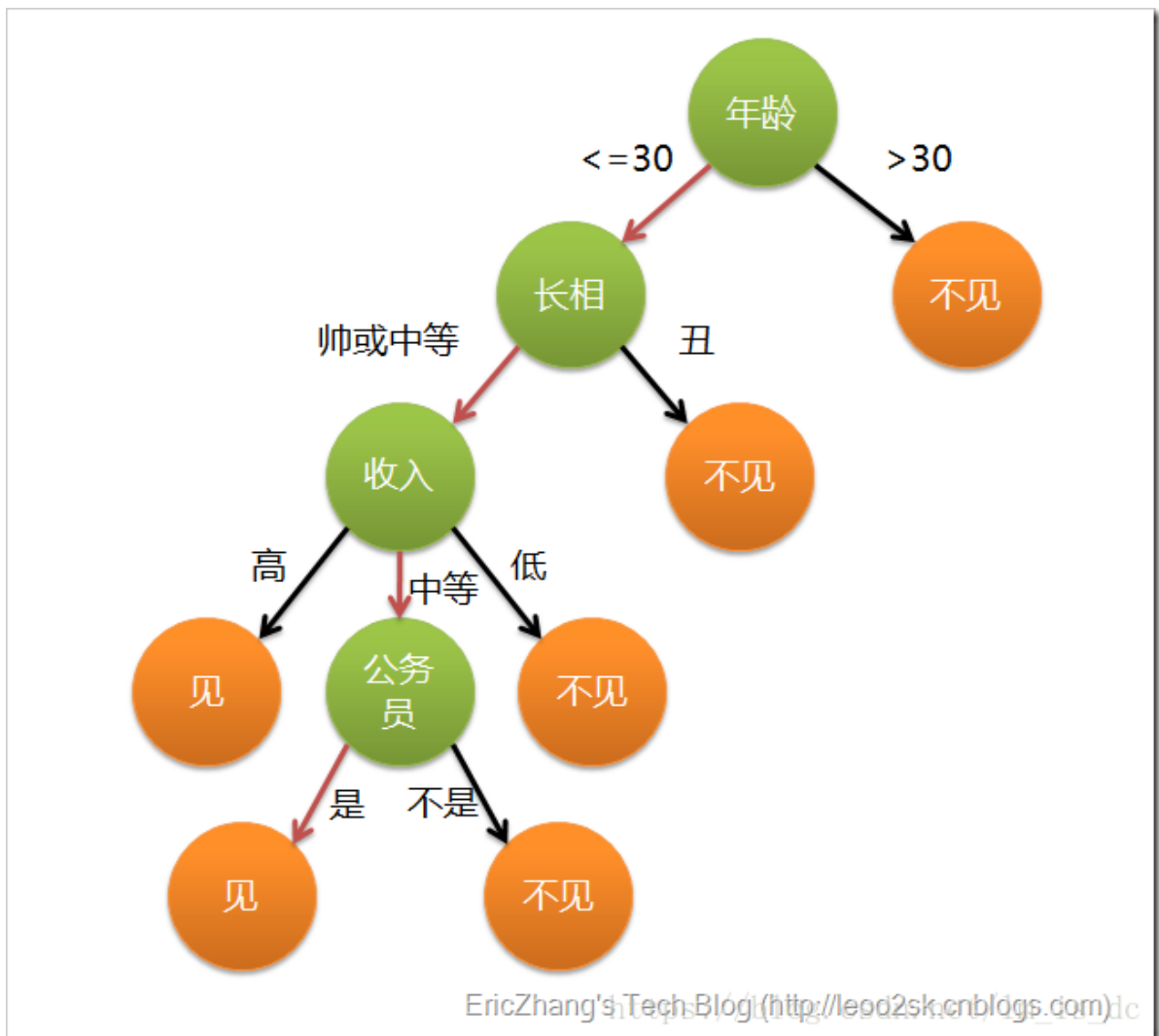
母亲：不算很高，中等情况。

女儿：是公务员不？

母亲：是，在税务局上班呢。

女儿：那好，我去见见。

这个女孩的决策过程就是典型的分类树决策。相当于通过年龄、长相、收入和是否公务员对将男人分为两个类别：见和不见。假设这个女孩对男人的要求是：30岁以下、长相中等以上并且是高收入者或中等以上收入的公务员，那么这个可以用下图表示女孩的决策逻辑：



上图完整表达了这个女孩决定是否见一个约会对象的策略，其中绿色节点表示判断条件，橙色节点表示决策结果，箭头表示在一个判断条件在不同情况下的决策路径，图中红色箭头表示了上面例子中女孩的决策过程。

这幅图基本可以算是一颗决策树，说它“基本可以算”是因为图中的判定条件没有量化，如收入高中低等等，还不能算是严格意义上的决策树，如果将所有条件量化，则就变成真正的决策树了。

有了上面直观的认识，我们可以正式定义决策树了：

决策树（decision tree）是一个**树结构**（可以是二叉树或非二叉树）。其每个非叶节点表示一个特征属性上的测试，每个分支代表这个特征属性在某个值域上的输出，而每个叶节点存放一个类别。使用决策树进行决策的过程就是从根节点开始，测试待分类项中相应的特征属性，并按照其值选择输出分支，直到到达叶子节点，将叶子节点存放的类别作为决策结果。

可以看到，决策树的决策过程非常直观，容易被人理解。目前决策树已经成功运用于**医学、制造产业、天文学、分支生物学以及商业**等诸多领域。

之前介绍的K-近邻算法可以完成很多分类任务，但是它最大的缺点就是无法给出数据的内在含义，**决策树的主要优势就在于数据形式非常容易理解。**

决策树算法能够读取数据集合，构建类似于上面的决策树。决策树很多任务都是为了数据中所蕴含的知识信息，因此决策树可以使用不熟悉的数据集合，并从中提取出一系列规则，机器学习算法最终将使用这些机器从数据集中创造的规则。专家系统中经常使用决策树，而且决策树给出结果往往可以匹敌在当前领域具有几十年工作经验的人类专家。

二、决策树的构造

分类解决离散问题，回归解决连续问题

- 决策树：信息论
- 逻辑斯蒂回归、贝叶斯：概率论

不同于逻辑斯蒂回归和贝叶斯算法，决策树的构造过程不依赖领域知识，它使用属性选择度量来选择将元组最好地划分成不同的类的属性。所谓决策树的构造就是进行属性选择度量确定各个特征属性之间的拓扑结构。

构造决策树的关键步骤是分裂属性。所谓分裂属性就是在某个节点处按照某一特征属性的不同划分构造不同的分支，其目标是让各个分裂子集尽可能地“纯”。尽可能“纯”就是尽量让一个分裂子集中待分类项属于同一类别。分裂属性分为三种不同的情况：

- 1、属性是离散值且不要求生成二叉决策树。
此时用属性的每一个划分作为一个分支。
- 2、属性是离散值且要求生成二叉决策树。
此时使用属性划分的一个子集进行测试，
按照“属于此子集”和“不属于此子集”分成两个分支。
- 3、属性是连续值。此时确定一个值作为分裂点`split_point`，
按照`>split_point`和`<=split_point`生成两个分支。

构造决策树的关键性内容是进行属性选择度量，属性选择度量是一种选择分裂准则，它决定了拓扑结构及分裂点`split_point`的选择。

属性选择度量算法有很多，一般使用自顶向下递归分治法，并采用不回溯的贪心策略。这里介绍常用的ID3算法。

ID3算法

划分数据集的大原则是：**将无序的数据变得更加有序。**

我们可以使用多种方法划分数据集，但是每种方法都有各自的优缺点。组织杂乱无章数据的一种方法就是使用信息论度量信息，信息论是量化处理信息的分支科学。我们可以在划分数据之前使用信息论量化度量信息的内容。

在划分数据集之前之后信息发生的变化称为信息增益，知道如何计算信息增益，我们就可以计算每个特征值划分数据集获得的信息增益，获得信息增益最高的特征就是最好的选择。

在可以评测哪种数据划分方式是最好的数据划分之前，我们必须学习如何计算信息增益。**集合信息的度量方式称为香农熵或者简称为熵**，这个名字来源于**信息论之父克劳德·香农**。

entropy

熵定义为信息的期望值，在明晰这个概念之前，我们必须知道信息的定义。如果待分类的事务可能划分在多个分类之中，则符号 x 的信息定义为：

$$I(x_i) = -\log_2 p(x_i)$$

其中 $p(x)$ 是选择该分类的概率。

为了计算熵，我们需要计算所有类别所有可能值包含的信息期望值，通过下面的公式得到：

$$H = -\sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

其中n是分类的数目。

在决策树当中，设D为用类别对训练元组进行的划分，则D的熵（entropy）表示为：

$$info(D) = - \sum_{i=1}^m p_i \log_2(p_i)$$

其中 p_i 表示第i个类别在整个训练元组中出现的概率，可以用属于此类别元素的数量除以训练元组元素总数量作为估计。**熵的实际意义表示是D中元组的类标号所需要的平均信息量。**

现在我们假设将训练元组D按属性A进行划分，则A对D划分的期望信息为：

$$info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} info(D_j)$$

而**信息增益**即为两者的差值：

$$gain(A) = info(D) - info_A(D)$$

ID3算法就是在每次需要分裂时，计算每个属性的增益率，然后选择增益率最大的属性进行分裂。下面我们继续用SNS社区中不真实账号检测的例子说明如何使用ID3算法构造决策树。为了简单起见，我们假设训练集合包含10个元素：

日志密度	好友密度	是否使用真实头像	账号是否真实
s	s	no	no
s	l	yes	yes
l	m	yes	yes
m	m	yes	yes
l	m	yes	yes
m	l	no	yes
m	s	no	no
l	m	no	yes
m	s	no	yes
s	s	yes	no

其中s、m和l分别表示小、中和大。

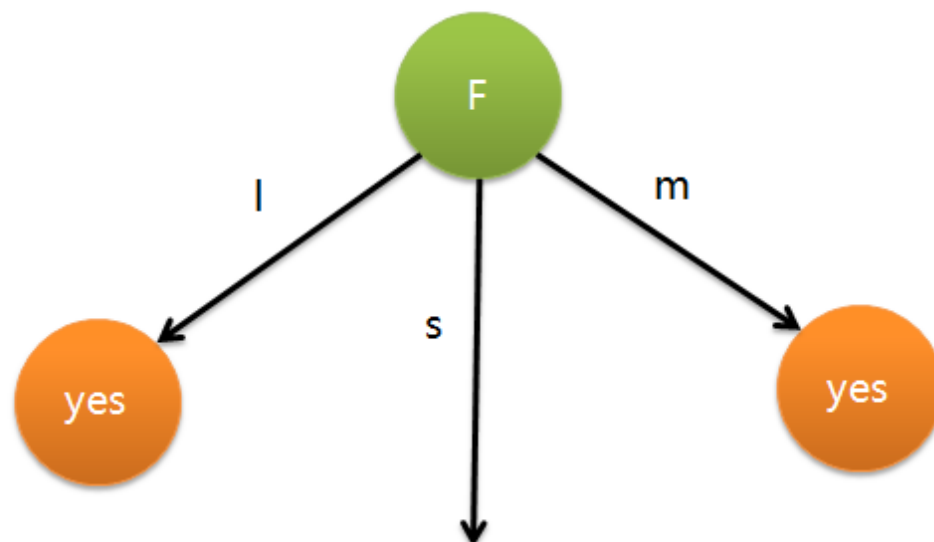
设L、F和H表示日志密度、好友密度、是否使用真实头像，下面计算各属性的信息增益。

$$\begin{aligned} info(D) &= -0.7 \log_2 0.7 - 0.3 \log_2 0.3 = 0.7 * 0.51 + 0.3 * 1.74 = 0.879 \\ info_L(D) &= 0.3 * \left(-\frac{0}{3} \log_2 \frac{0}{3} - \frac{3}{3} \log_2 \frac{3}{3} \right) + 0.4 * \left(-\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4} \right) + 0.3 * \\ &\quad \left(-\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} \right) = 0 + 0.326 + 0.277 = 0.603 \\ gain(L) &= 0.879 - 0.603 = 0.276 \end{aligned}$$

因此日志密度的信息增益是0.276。

用同样方法得到F和H的信息增益分别为0.553和0.033。

因为F具有最大的信息增益，所以第一次分裂选择F为分裂属性，分裂后的结果如下图所示：



日志密度	是否使用真实头像	账号是否真实
s	no	no
m	no	no
m	no	yes
s	yes	no

https://blog.csdn.net/lm_is_dc

在上图的基础上，再递归使用这个方法计算子节点的分裂属性，最终就可以得到整个决策树。

注意：以下命令都是在浏览器中输入。

cmd命令窗口输入：jupyter notebook

打开浏览器输入网址<http://localhost:8888/>

上图的信息熵，确定下一个分类的特征

```
import numpy as np
```

```
# 账号是否真实的信息熵：  
# 3个no (0.3) 7个yes (0.7)
```

```
info_D = -(0.3*np.log2(0.3) + 0.7*np.log2(0.7))  
info_D
```

blog.csdn.net/lm_is_dc
0.8812908992306927

```
# 以好友密度进行划分  
# 概率  
# 4个s(0.4) ----> 3个no(0.75) 1个yes(0.25)  
# 4个m(0.4) ----> 4个yes(1)  
# 2个l(0.2) ----> 2个yes(1)
```

```
info_F_D = -(0.4*(0.75*np.log2(0.75) + 0.25*np.log2(0.25)) + 0.4*(1*np.log2(1)) +
0.2*(1*np.log2(1)))
info_F_D
```

0.32451124978365314

```
# 日志密度进行划分，熵
# 概率
# 3个s(0.3) --->2个no(2/3) 1个yes(1/3)
# 4个m(0.4) --->1个no(0.25) 3个yes(0.75)
# 3个l(0.3) --->3个yes(1)

info_L_D = -(0.3*(0.667*np.log2(0.667) + 0.334*np.log2(0.334))
+ 0.4*(0.25*np.log2(0.25) + 0.75*np.log2(0.75))
+0.3*(1*np.log2(1)))

info_L_D
```

0.5999423558297423

```
# 信息增益
# 以好友密度进行划分
info_D - info_F_D
```

0.55677964944703962

```
# 信息增益
# 以日志密度进行划分
info_D - info_L_D
```

0.28134854340095039

分析：0.556>0.28，所以第一次分裂选择F为分裂属性。

三、实战

【注意】参数max_depth越大，越容易过拟合。

1、使用自带的iris数据集

####1.1 使用决策树算法

```
from IPython.display import display
#导入决策树包
from sklearn.tree import DecisionTreeClassifier
```

```
import sklearn.datasets as datasets
```

```
iris = datasets.load_iris()
x = iris.data
y = iris.target
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,
                                                  test_size = 0.3)
```

```
display(X_train.shape,X_test.shape)
```

(105, 4)

(45, 4)

鸢尾花数据集一共有150个样本，每个样本的特征(属性)有4个。

```
#创建决策树模型，深度为3
tree = DecisionTreeClassifier(max_depth=3)
#训练模型
tree.fit(X_train,y_train)

# 算法获得的结果:y_、cofe_(系数)、intercepter_(截距)
#预测结果返回值变量命名一般带 '_'
y_ = tree.predict(X_test)

display(y_test,y_)
```

```
array([1, 2, 2, 2, 2, 0, 0, 1, 1, 0, 1, 0, 2, 0, 2, 1, 0, 0, 1, 0, 2, 0, 1,
       0, 0, 0, 0, 0, 1, 0, 2, 0, 2, 2, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1])
array([1, 2, 2, 2, 2, 0, 0, 1, 1, 0, 1, 0, 2, 0, 2, 2, 0, 0, 1, 0, 2, 0, 1,
       0, 0, 0, 0, 0, 1, 0, 2, 0, 2, 2, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1])
```

```
#决策树模型使用测试数据进行评分
tree.score(X_test,y_test)
```

0.9777777777777775

1.2 使用KNN算法

```
#导包
from sklearn.neighbors import KNeighborsClassifier
```

```
#创建knn模型，使用5个点(邻居)做比较
knn = KNeighborsClassifier(n_neighbors=5)
#训练模型
knn.fit(X_train,y_train)
#knn模型使用测试数据进行评分
knn.score(X_test,y_test)
```

log.csdn.net/ln_is_dc
0.9555555555555556

1.3 使用逻辑斯蒂回归算法

```
#导包
from sklearn.linear_model import LogisticRegression

logistic = LogisticRegression()

logistic.fit(X_train,y_train)

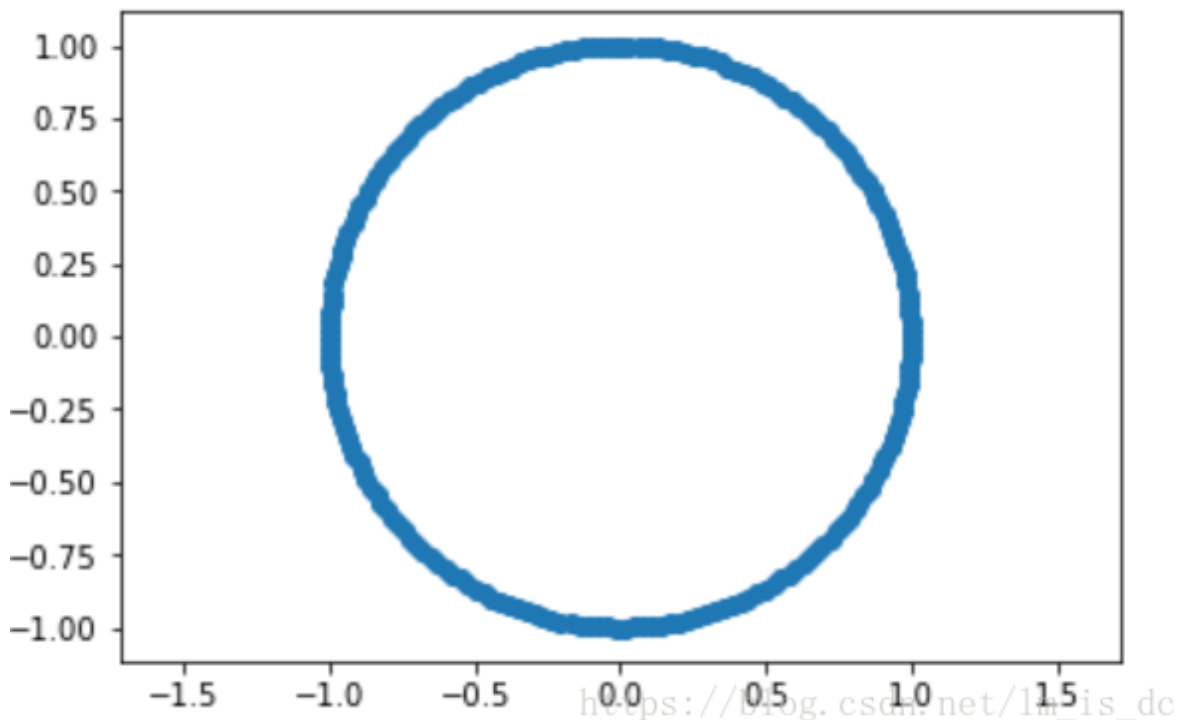
logistic.score(X_test,y_test)
```

log.csdn.net/ln_is_dc
0.9555555555555556

2、使用回归预测一个椭圆

```
import matplotlib.pyplot as plt
# 魔法指令, plt.show(),可以省略了
%matplotlib inline
```

```
# 角度值2pi = 360
# np.pi
# 训练数据, 格式二维
x = np.linspace(0,2*np.pi,200).reshape(-1,1)
y = np.concatenate((np.sin(x),np.cos(x)),axis=1)
plt.scatter(y[:,0],y[:,1])
plt.axis('equal')
```

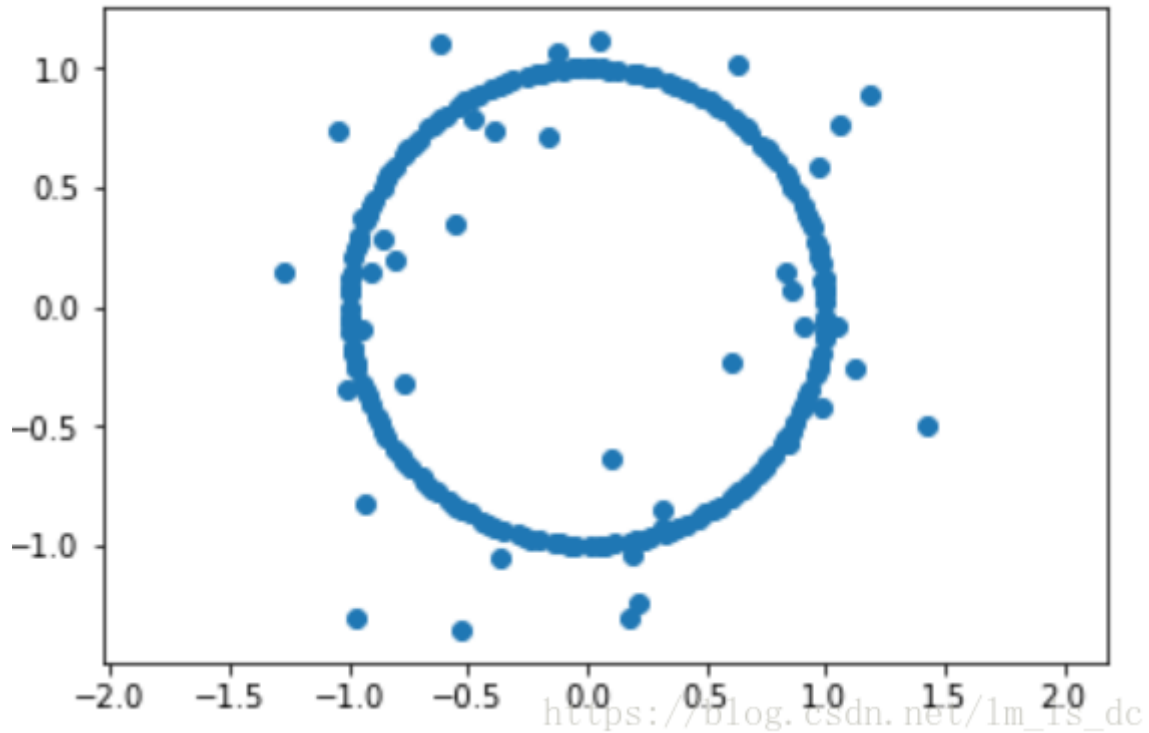


添加噪声:

```
y[:,5].shape
```


t(40, 2)dc

```
#添加噪声
y[:,5] += np.random.randn(40,2)*0.3
plt.scatter(y[:,0],y[:,1])
plt.axis('equal')
```



创建不同深度的决策树

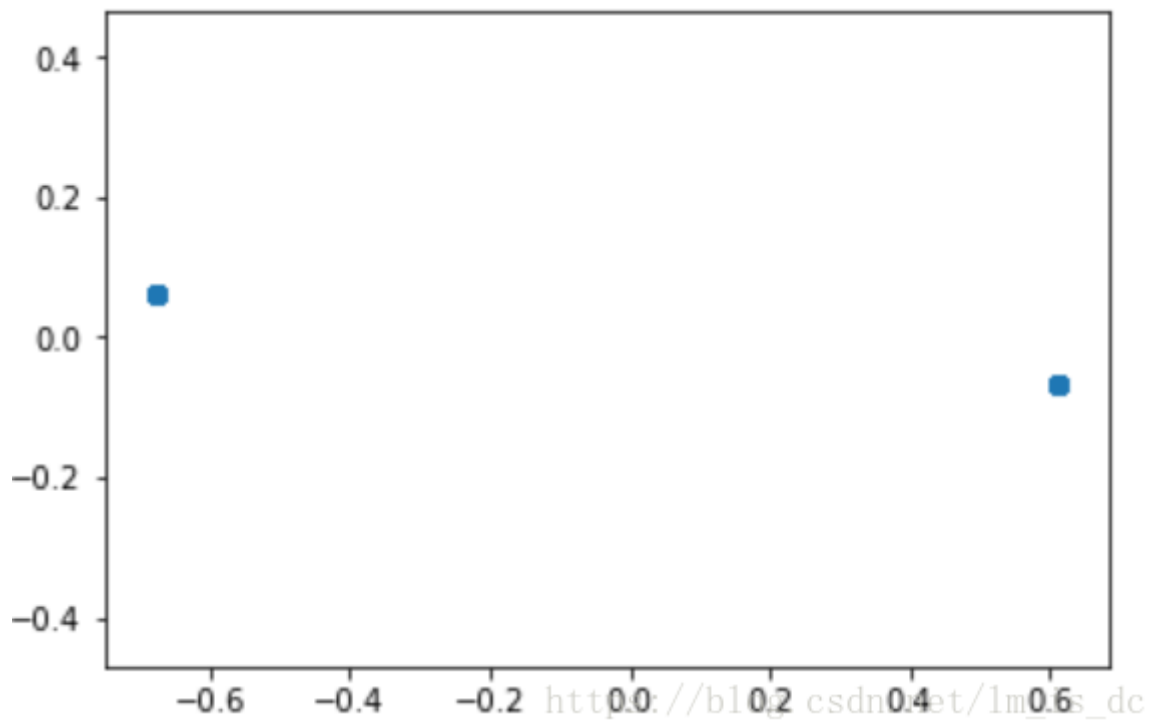
进行数据训练

```
from sklearn.tree import DecisionTreeRegressor
```

深度为1:

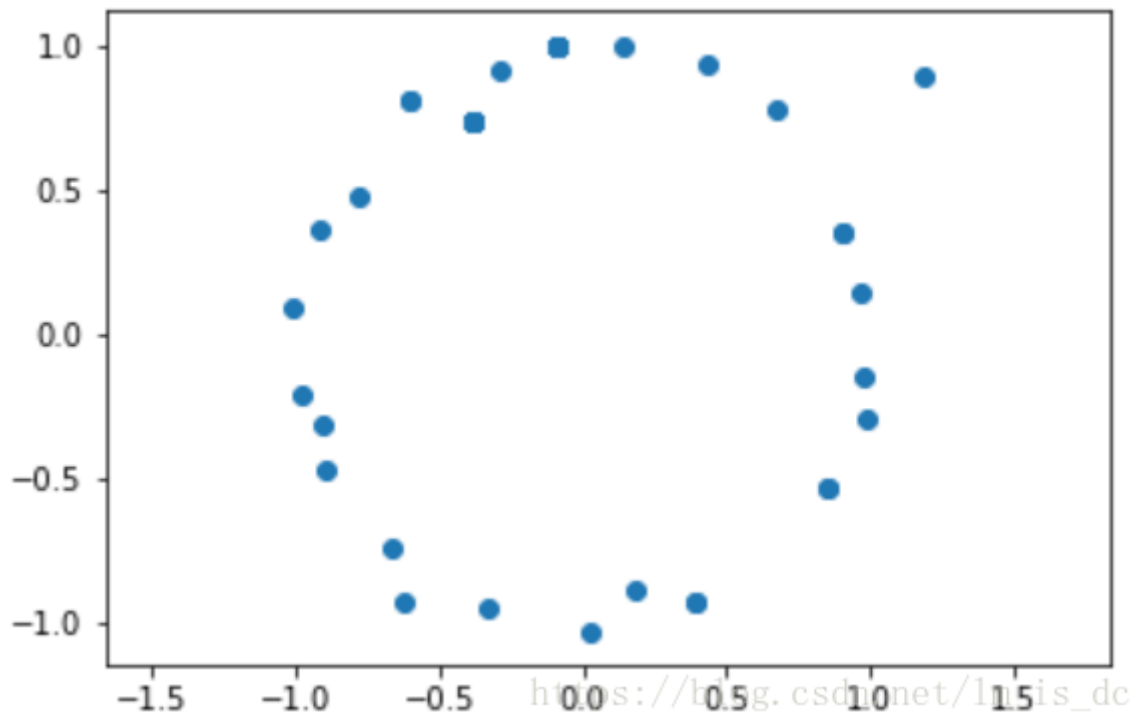
```
tree = DecisionTreeRegressor(max_depth=1)
tree.fit(X,y)
# 训练数据必须是二维的[[样本一],[样本二],[样本三].....]
X_test = np.linspace(-100,100,300).reshape(-1,1)
# 目标值y ,可以是二维的或者多维

X_test = np.linspace(-100,100,900).reshape(-1,1)
y_ = tree.predict(X_test)
plt.scatter(y_[:,0],y_[:,1])
plt.axis('equal')
```



深度为5:

```
tree = DecisionTreeRegressor(max_depth=5)
tree.fit(X,y)
X_test = np.linspace(-100,100,900).reshape(-1,1)
y_ = tree.predict(X_test)
plt.scatter(y_[::0],y_[::1])
plt.axis('equal')
```

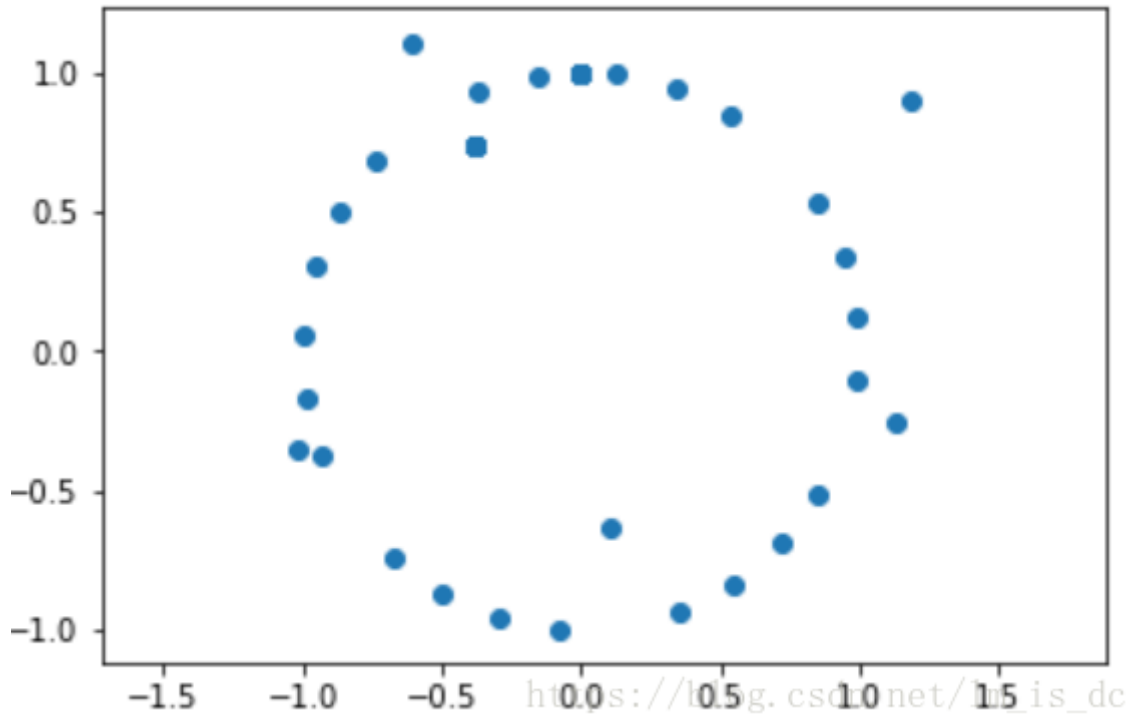


深度为100:

```

tree = DecisionTreeRegressor(max_depth=100)
tree.fit(X,y)
X_test = np.linspace(-100,100,900).reshape(-1,1)
y_ = tree.predict(X_test)
plt.scatter(y_[0],y_[1])
plt.axis('equal')

```



##四、随机森林

tree 一棵树

Forest森林（多棵树）

RandomForestClassifier用到的算法依然是决策树，

RandomForestClassifier将训练数据以随机抽样的方式（重复），抽取多次，多次数据进行训练，求解平均值。

```

#导入包
from sklearn.ensemble import RandomForestClassifier
import sklearn.datasets as datasets
iris = datasets.load_iris()

```

```
#获取鸢尾花特征数据
x = iris.data
#获取目标数据
y = iris.target
#把样本数据随机分成训练数据和测试数据
X_train,X_test,y_train,y_test = train_test_split(x,y,
                                                    test_size = 0.3)

#创建数据森林模型
forest = RandomForestClassifier()
#训练数据
forest.fit(X_train,y_train)
#评分
forest.score(X_test,y_test)
```

: 0.9555555555555556 is_dc